LAPRAK TEKNOLOGI IOT

TUGAS PRAKTIKUM 8

Oleh:

NAMA : ERVAN GUSTI WANJAYA

NPM : 25782096

KELAS : TRI – 3D

Dosen:

Dr. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd.,M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

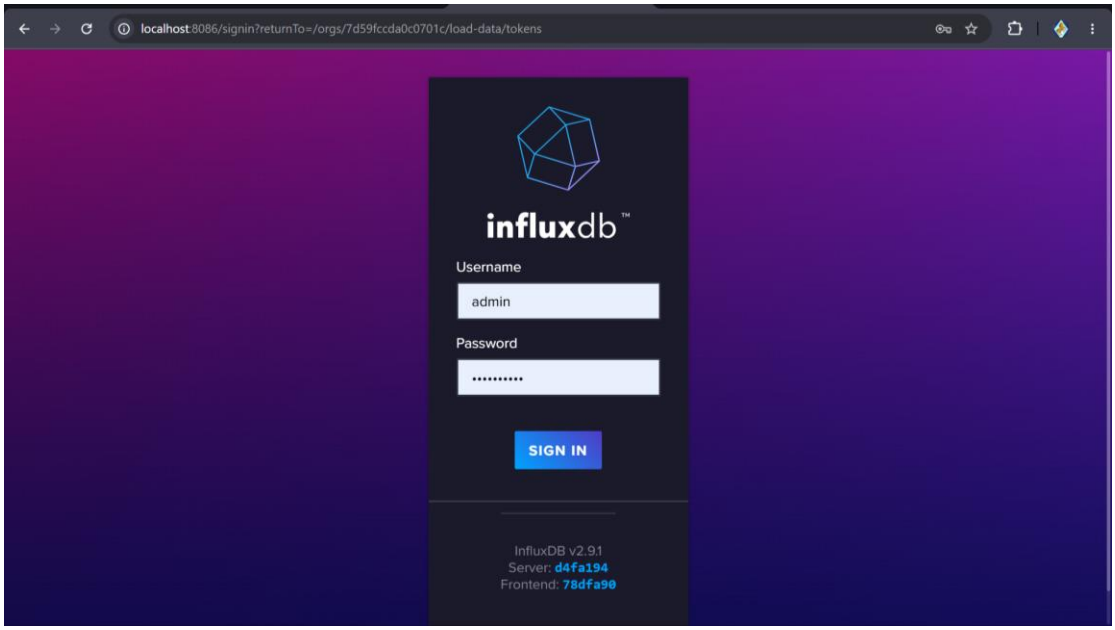
2025/2026

8.5.1 Praktikum 1: Peningkatan Ekosistem IoT Server dengan InfluxDB v2 dan Grafana pada Docker Compose

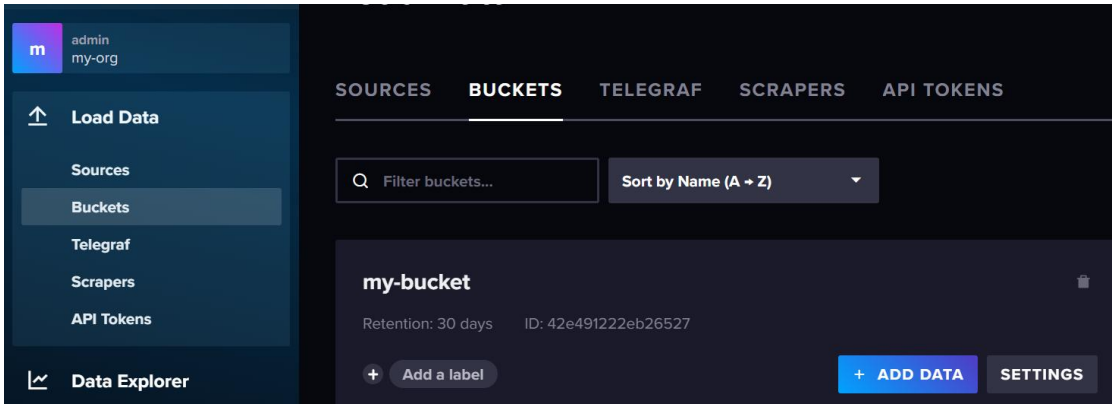
```
D:\iot-server>docker compose up -d
[+] up 4/4
 ✔Container influxdb2   Started                                                    0.3s
 ✔Container mqtt-broker Started                                                    0.6s
 ✔Container grafana     Started                                                    0.5s
 ✔Container node-red    Started                                                    0.5s

D:\iot-server>docker compose ps
NAME          IMAGE                       COMMAND                  SERVICE    CREATED      STATUS
     PORTS
grafana       grafana/grafana:latest      "/run.sh"                grafana    3 days ago   Up 10 seconds
     0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
influxdb2     influxdb:2                  "/entrypoint.sh infl…"   influxdb   3 days ago   Up 11 seconds
     0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
mqtt-broker   eclipse-mosquitto:latest    "/docker-entrypoint.…"   mqtt       3 days ago   Up 11 seconds
     0.0.0.0:1883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp
node-red      nodered/node-red:latest     "./entrypoint.sh"        nodered    3 days ago   Up 10 seconds (health: startin
g)   0.0.0.0:1880->1880/tcp, [::]:1880->1880/tcp
```

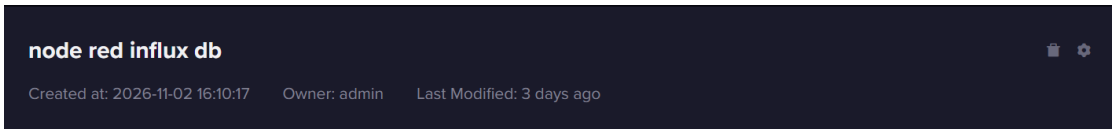
8.5.2 Praktikum 2: Inisialisasi dan Manajemen Kredensial & API Token InfluxDB v2



Memverifikasi Bucket Penyimpanan:

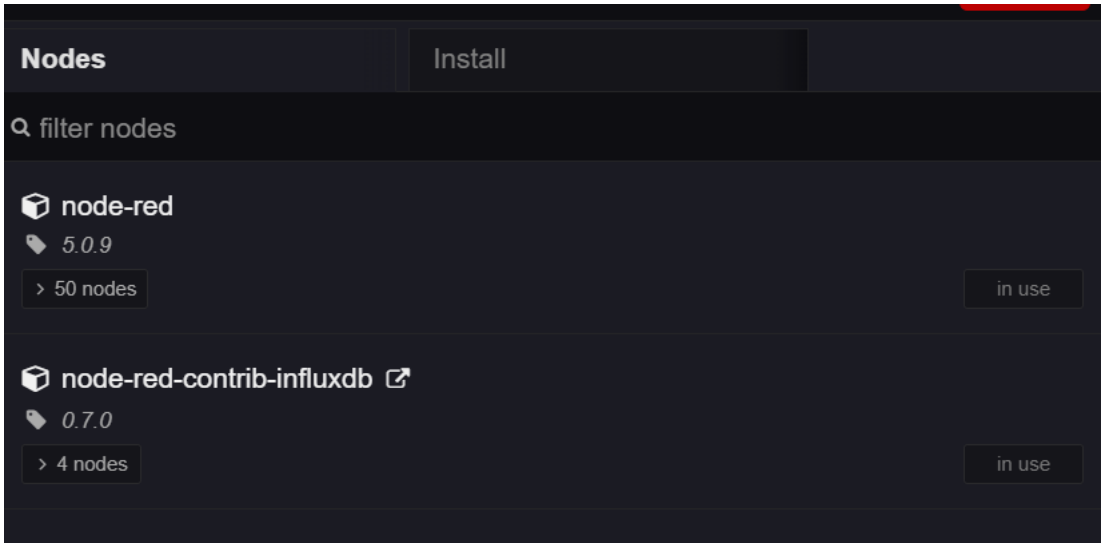


Menerbitkan Token Khusus untuk Node-RED (*Generate Scoped API Token*):

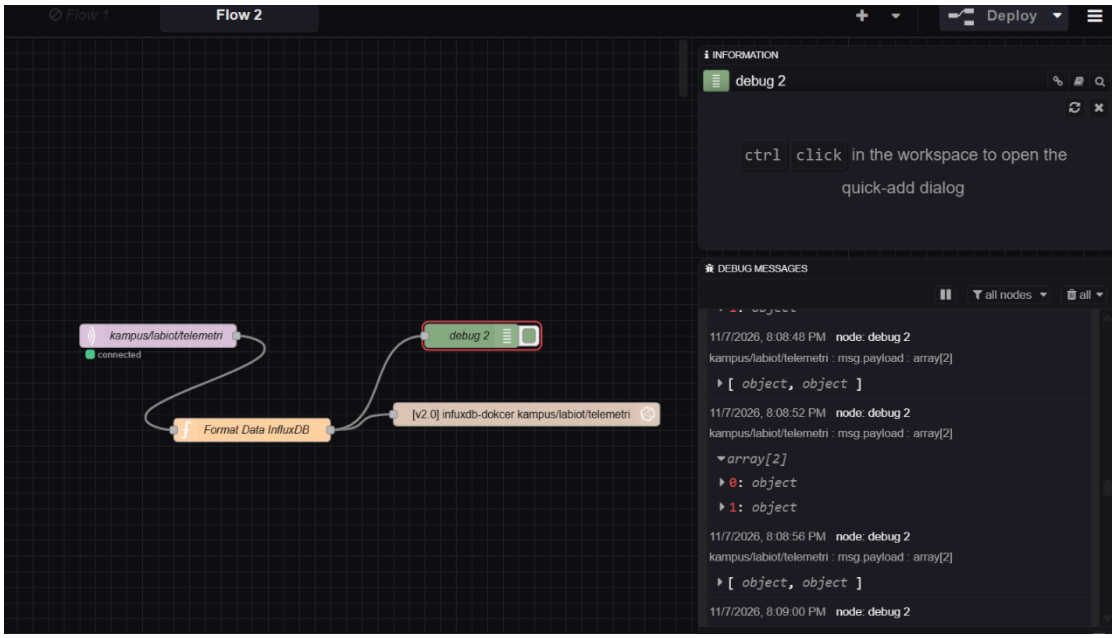


8.5.3 Praktikum 3: Instalasi Node InfluxDB dan Perancangan Pipeline Data pada Node-RED

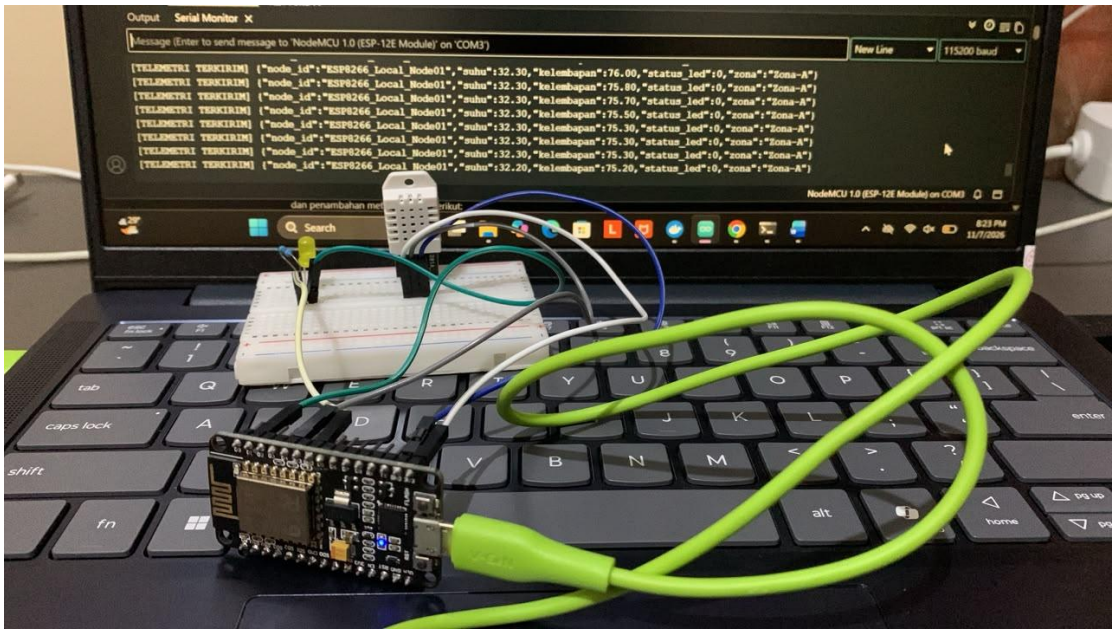
Memasang Modul node-red-contrib-influxdb



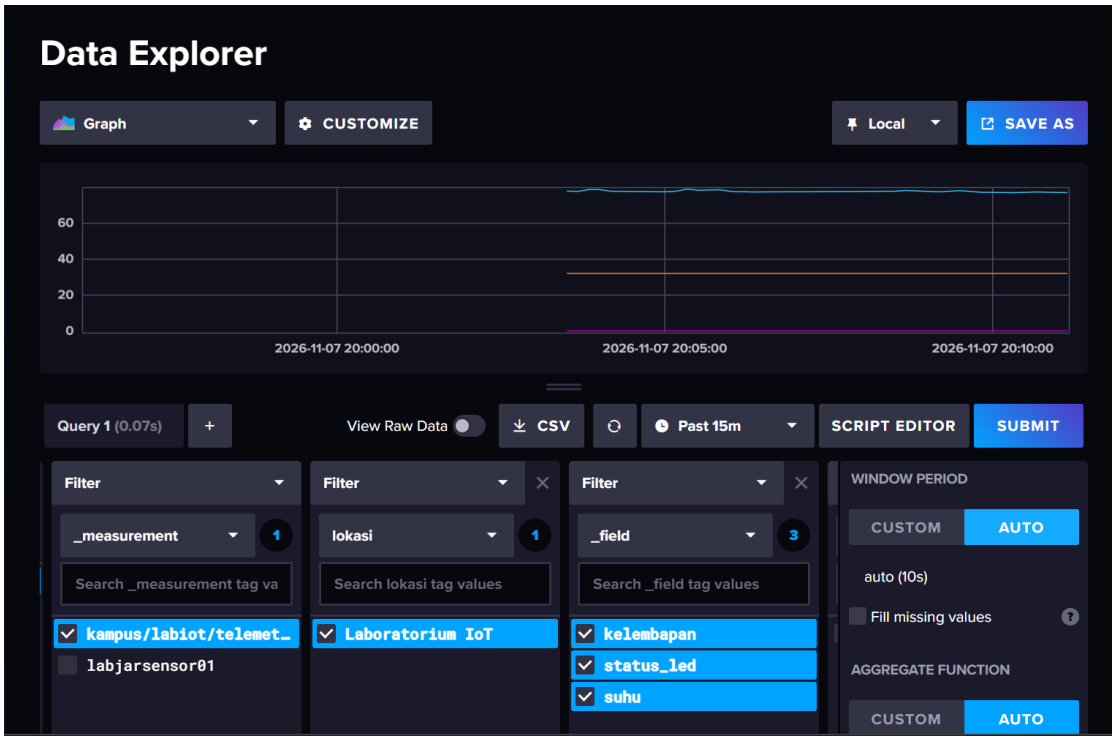
Alur Data (*Flow Pipeline*):

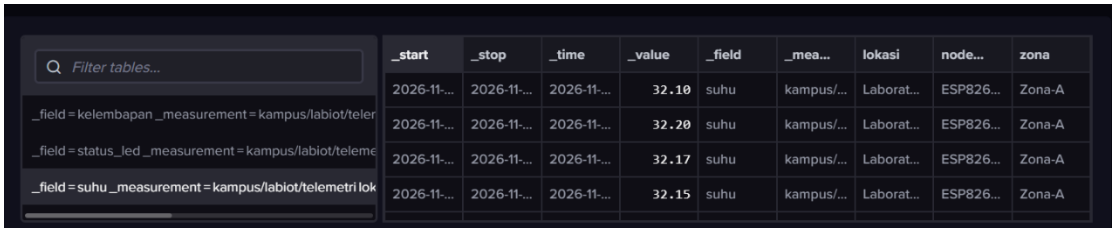


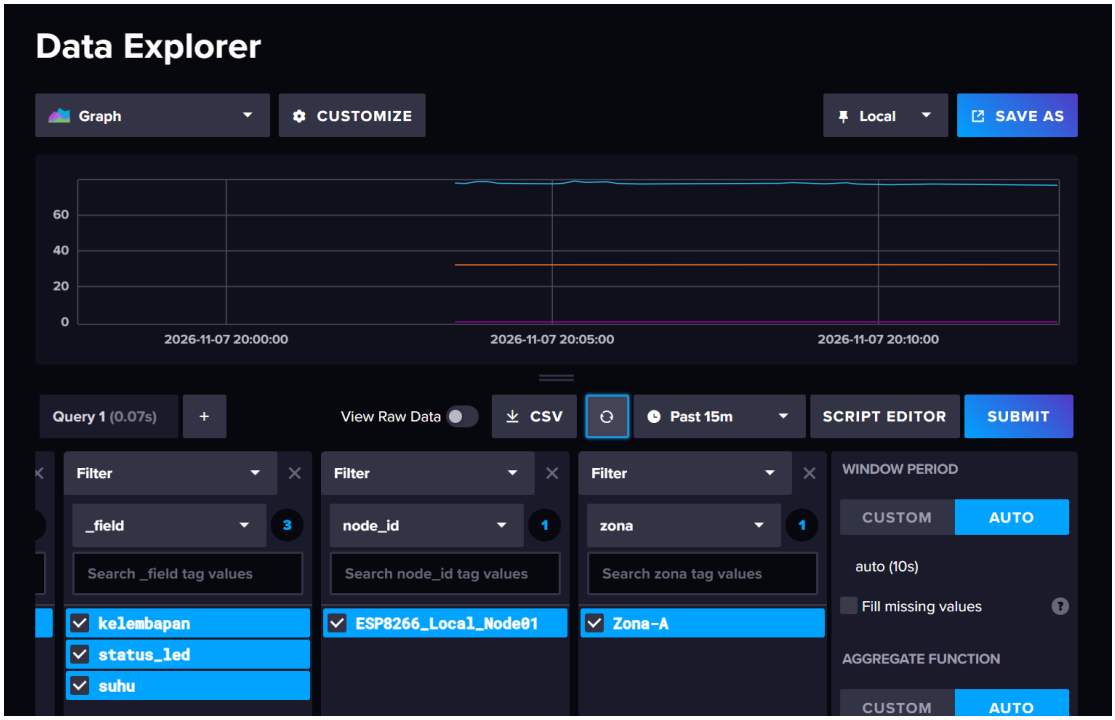
8.5.4 Praktikum 4: Integrasi End-to-End NodeMCU ESP8266 dan Verifikasi Telemetri Time-Series

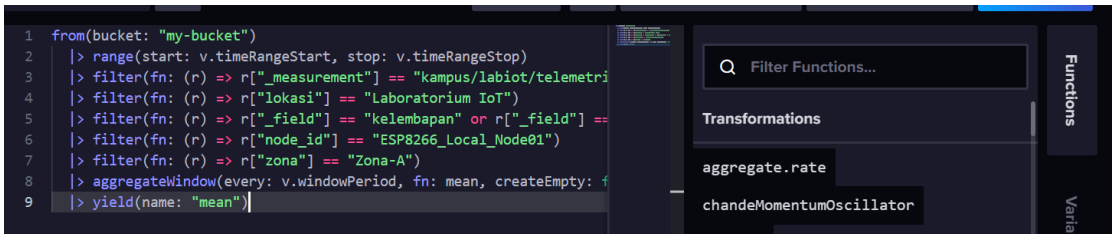


Verifikasi dan Analisis Data pada InfluxDB Data Explorer:









Latihan 1: Analisis Kinerja & Efisiensi Penyimpanan (TSDB vs RDBMS)

1. Perhitungan Jumlah Total Titik Data (Data Points)

- Frekuensi pengiriman: 1 titik data per 2 detik (setiap titik memuat 3 *field* float).
- Data per menit: 60 detik / 2 detik = 30 titik data per menit.
- Data per hari: 30 × 60 menit × 24 jam = 43.200 titik data per hari untuk 1 simpul.
- Total 30 hari untuk 100 simpul: 43.200 × 30 hari × 100 simpul = 129.600.000 titik data (data points).

2. Rasio Penghematan Ruang Penyimpanan (Storage Savings)

- Kebutuhan RDBMS (MySQL/PostgreSQL): 129.600.000 × 150 byte = 19.440.000.000 byte (~19,44 GB).
- Kebutuhan InfluxDB v2: 129.600.000 × 2,5 byte = 324.000.000 byte (~324 MB).
- Rasio Penghematan: (150 byte / 2,5 byte) = 60 : 1. InfluxDB 60 kali lebih efisien atau menghemat sekitar 98,33% kapasitas penyimpanan dibandingkan RDBMS tradisional.

3. Penurunan Performa (Write Degradation) pada Indeks B-Tree RDBMS

Indeks B-Tree dirancang untuk mempercepat pencarian data (*read-heavy*), bukan untuk laju penulisan konstan yang ekstrem (*write-heavy*). Saat jutaan baris data baru ditambahkan terus-menerus setiap detik, RDBMS harus terus melakukan penyeimbangan ulang struktur pohon (*tree rebalancing*), pemisahan *node* (*node splitting*), dan mengalami fragmentasi memori sekunder. *Overhead* operasi ini membebani siklus I/O disk sehingga menyebabkan antrean penulisan (*ingestion bottleneck*) dan penurunan performa database secara drastis.

Latihan 2: Fenomena Cardinality Explosion pada InfluxDB Tags

1. Mekanisme Inverted Index dan Pemicu Cardinality Explosion

Setiap nilai string yang dimasukkan ke dalam *Tag Set* akan diindeks oleh InfluxDB menggunakan struktur *Inverted Index* agar pencarian berbasis *tag* menjadi sangat cepat (O(1)). *Cardinality* adalah jumlah kombinasi unik dari seluruh *Tag*. Jika pengembang memasukkan nilai berpresisi tinggi atau acak yang terus berubah (seperti UUID transaksi atau angka desimal sensor), InfluxDB akan membuat indeks baru untuk setiap titik data. Hal ini memicu *Cardinality Explosion*, di mana jumlah indeks unik meledak tak terbatas sejalan dengan jumlah data yang masuk.

2. Dampak Fatal Terhadap RAM dan Stabilitas Docker

Seluruh struktur *Inverted Index* disimpan di dalam memori utama (RAM) agar kueri berkecepatan tinggi dapat dilakukan. Terjadinya *Cardinality Explosion* akan membuat InfluxDB memakan seluruh kapasitas RAM yang tersedia (*memory leak*). Ketika RAM habis, sistem operasi akan memicu *Out of Memory (OOM) Killer*, yang secara paksa mematikan kontainer Docker influxdb (status *OOMKilled*), menyebabkan sistem mati (*downtime*) dan berpotensi merusak *file* penyimpanan data (korup).

3. Pedoman Baku: Kapan Menggunakan Tag vs Field

- Gunakan TAG (Diindeks): Hanya untuk metadata diskrit yang nilainya terbatas (finit), relatif statis, dan digunakan untuk pengelompokan (GROUP BY) atau pemfilteran (WHERE). Contoh: lokasi, id_perangkat, zona, versi_firmware.
- Gunakan FIELD (Tidak Diindeks): Untuk seluruh data metrik pengukuran fisik yang nilainya selalu berubah (*unbound*), angka presisi, atau tipe data *string* acak/panjang. Contoh: suhu, kelembapan, uuid_pesan, pesan_log.

Latihan 3: Arsitektur Jaringan Docker dan Resolusi Nama Layanan DNS

1. Penyebab Kegagalan Koneksi via localhost di Dalam Node-RED

Kontainer Docker beroperasi secara terisolasi. Jika Anda menuliskan URL http://localhost:8086 di dalam konfigurasi node InfluxDB Out pada Node-RED, aplikasi Node-RED akan mencari port 8086 di dalam kontainernya sendiri (layanan node-red), bukan di laptop fisik (host) atau di kontainer InfluxDB. Karena tidak ada database InfluxDB yang berjalan di dalam kontainer Node-RED, koneksi pasti ditolak (*Connection Refused*).

2. Peran Docker Embedded DNS dan Jaringan Bridge

Jaringan *bridge* (iot-net) menciptakan satu jaringan lokal area virtual yang menghubungkan semua kontainer di dalam berkas docker-compose.yml. *Docker Embedded DNS Resolver* bertugas menerjemahkan nama layanan (misal: influxdb atau mqtt) menjadi alamat IP internal kontainer secara dinamis. Dengan mekanisme ini, kontainer Node-RED cukup menggunakan URL http://influxdb:8086 untuk berkomunikasi dengan InfluxDB tanpa perlu membuka *port* tersebut ke internet publik, sehingga menjamin keamanan dan isolasi sistem (*port 8086* hanya perlu di-*publish* jika pengguna ingin mengakses antarmuka *web* dari *browser* host).

Latihan 4: Strategi Retensi Data & Agregasi Berjenjang (Downsampling)

1. Inefisiensi Penyimpanan Data Presisi Detik Jangka Panjang

Nilai informasi dari telemetri presisi detik sangat berharga pada saat kejadian (untuk melacak *root cause analysis*), namun menyusut drastis setelah beberapa minggu (*data aging*). Menyimpan data presisi detik selama bertahun-tahun sangat boros *disk storage* dan akan memperlambat proses analitik. Untuk menampilkan grafik tren satu tahun, dasbor analitik akan dipaksa merender dan menghitung miliaran titik data yang tidak relevan secara visual, sehingga kueri menjadi lambat atau bahkan *timeout*.

2. Konsep Retention Policy pada Bucket InfluxDB v2*Retention Policy* adalah mekanisme penghapusan data otomatis (*auto-expiry*) bawaan dari InfluxDB. Ketika masa retensi pada sebuah *bucket* telah dikonfigurasi (misalnya 7 hari), sistem akan mengevaluasi usia cap waktu (*timestamp*) setiap baris data di *background*. Begitu data melewati batas usia 7 hari, InfluxDB akan menghapusnya dari level sistem berkas secara efisien, tanpa memerlukan kueri instruksi DELETE yang memberatkan kinerja I/O disk.

3. Rancang Bangun Arsitektur Dual-Bucket Strategy (Flux Task)

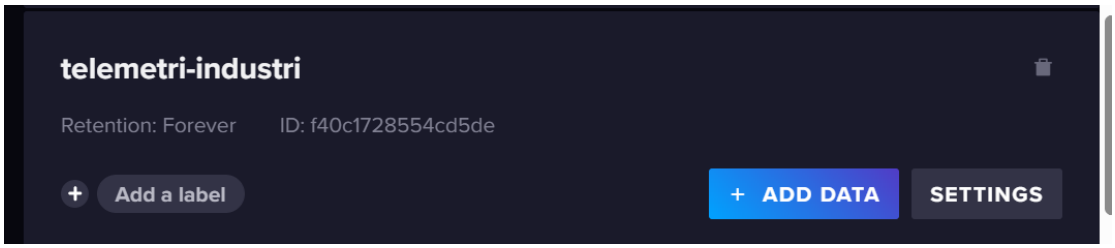
- Setup Arsitektur: Buat dua *bucket*: bucket_raw_7d (diatur retensi 7 hari untuk menyimpan data per detik) dan bucket_aggregated_1y (diatur retensi 365 hari untuk menyimpan data rangkuman).
- Mekanisme Skrip Flux: Buat InfluxDB Task (*cron-job* internal) yang dijalankan setiap jam sekali.
- Alur Kueri Flux:

    1. Tarik data dari *bucket* sumber (from(bucket: "bucket_raw_7d")).
    2. Ambil rentang waktu satu jam terakhir (range(start: -1h)).
    3. Lakukan penyusutan data berdasarkan rata-rata tiap jam (aggregateWindow(every: 1h, fn: mean)).
    4. Simpan hasil rata-rata tersebut ke *bucket* jangka panjang (to(bucket: "bucket_aggregated_1y")).

- Hasil: Manajemen dapat melihat tren tahunan yang sangat ringan dari bucket_aggregated_1y, sementara miliaran data sampah berpresisi tinggi di bucket_raw_7d akan bersih otomatis dengan sendirinya.

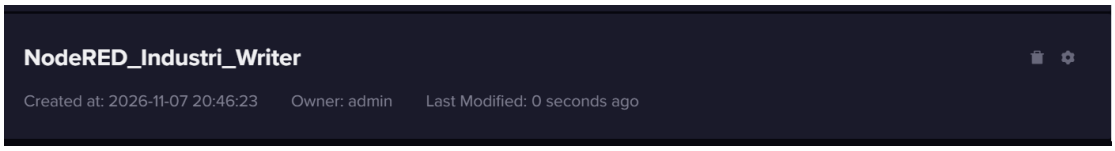
8.7 Tugas: Implementasi Metadata Tagging Multi-Zona dan Pemfilteran Kueri Telemetri

Pembuatan Bucket Khusus Industri di InfluxDB v2:

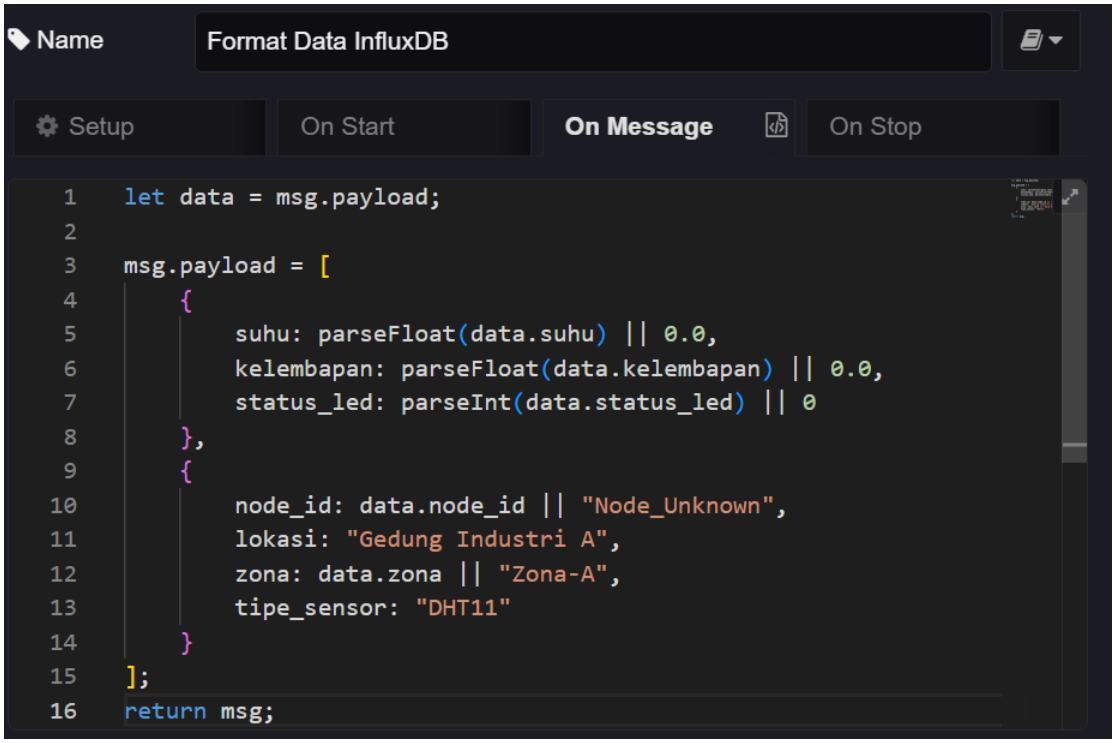


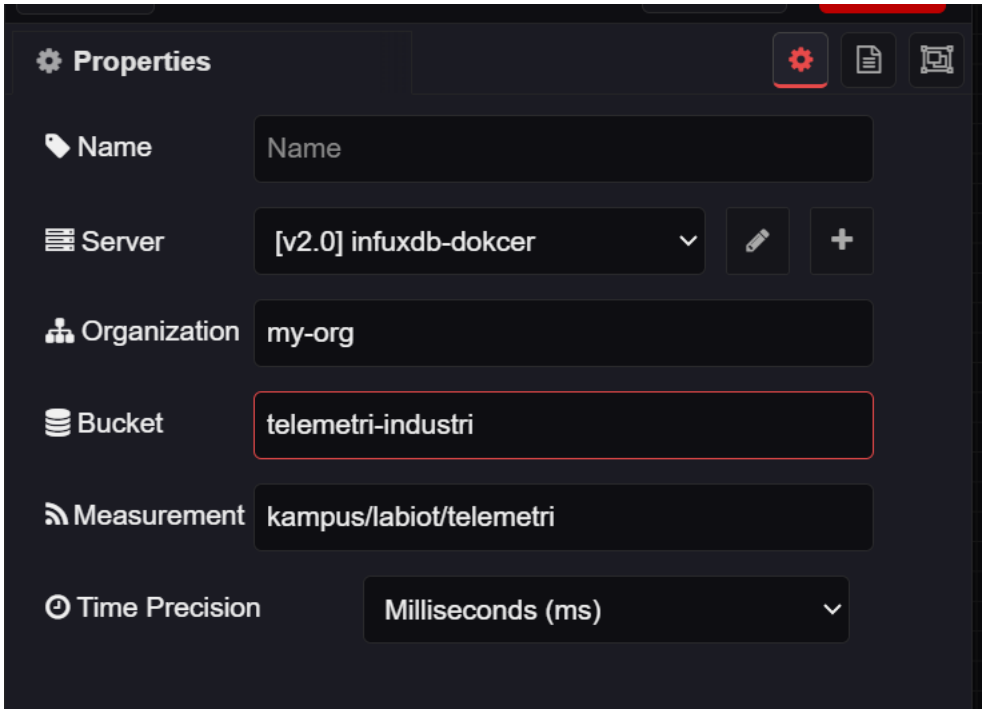


Pembaruan API Token Khusus:



Rekayasa Alur Kerja pada Node-RED:





Perancangan Skrip Kueri Flux Analitik Lanjut:

